Zara Ahmed

Fa25-BCS-060

TASK#01

```
#include <iostream>
#include <string>
using namespace std;

struct Node
{
   char data;
   Node* next;
};

class Stack
{
private:
   Node* top;

public:
   Stack()
   {
      top = NULL;
   }

   void push(char value)
   {
```

```
        Node* p = new Node;
        p->data = value;
        p->next = top;
        top = p;
    }

    char pop()
    {
        if (top == NULL)
            return '\0';

        Node* p = top;
        char value = p->data;
        top = top->next;
        delete p;

        return value;
    }

    bool isEmpty()
    {
        return top == NULL;
    }
};

bool isBalanced(string expression)
```

```
{
    Stack s;

    for (int i = 0; i < expression.length(); i++)
    {
        char ch = expression[i];

        if (ch == '(' || ch == '[' || ch == '{')
        {
            s.push(ch);
        }
        else if (ch == ')' || ch == ']' || ch == '}')
        {
            if (s.isEmpty())
                return false;

            char top = s.pop();

            if ((ch == ')' && top != '(') ||
                (ch == ']' && top != '[') ||
                (ch == '}' && top != '{'))
            {
                return false;
            }
        }
    }
```

```
    return s.isEmpty();
}

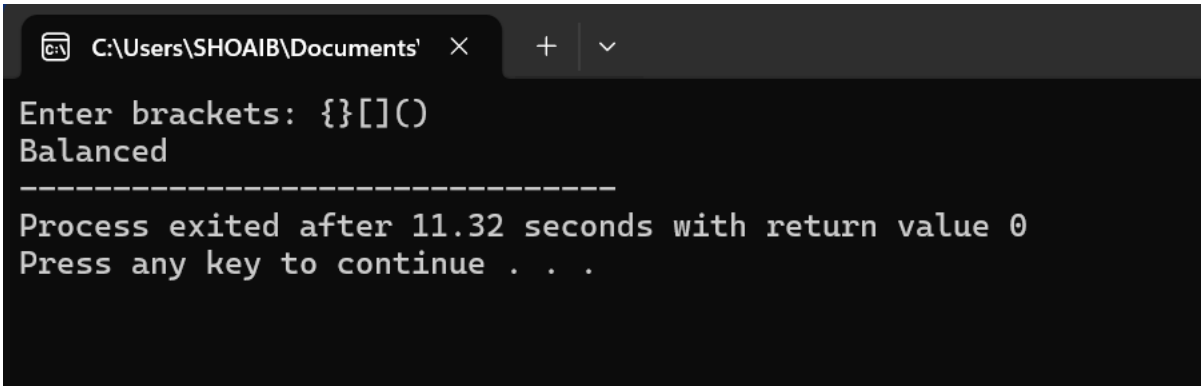
int main()
{
    string expression;

    cout << "Enter brackets: ";
    getline(cin, expression);

    if (isBalanced(expression))
        cout << "Balanced";
    else
        cout << "Not Balanced";

    return 0;
}
```

OUTPUT:



TASK#02

```
#include <iostream>
#include <string>
using namespace std;

struct Node
{
   char data;
   Node* next;
};

class Stack
{
private:
   Node* top;

public:
   Stack()
   {
      top = NULL;
   }

   void push(char value)
   {
      Node* p = new Node;
      p->data = value;
```

```
    p->next = top;
    top = p;
}

char pop()
{
    if (top == NULL)
        return '\0';

    Node* p = top;
    char value = p->data;
    top = top->next;
    delete p;

    return value;
}

char peek()
{
    if (top == NULL)
        return '\0';

    return top->data;
}

bool isEmpty()
```

```
    {
        return top == NULL;
    }
};

int priority(char op)
{
    if (op == '^')
        return 3;

    if (op == '*' || op == '/')
        return 2;

    if (op == '+' || op == '-')
        return 1;

    return 0;
}

string infixToPostfix(string infix)
{
    Stack s;
    string postfix = "";

    for (int i = 0; i < infix.length(); i++)
    {
```

```
char ch = infix[i];

if (ch == ' ')
   continue;

if ((ch >= 'A' && ch <= 'Z') ||
   (ch >= 'a' && ch <= 'z') ||
   (ch >= '0' && ch <= '9'))
{
   postfix += ch;
}

else if (ch == '(')
{
   s.push(ch);
}

else if (ch == ')')
{
   while (!s.isEmpty() && s.peek() != '(')
   {
      postfix += s.pop();
   }

   if (!s.isEmpty())
      s.pop();
```

```
        }

        else
        {
            while (!s.isEmpty() &&
                   priority(s.peek()) >= priority(ch))
            {
                postfix += s.pop();
            }

            s.push(ch);
        }
    }

    while (!s.isEmpty())
    {
        postfix += s.pop();
    }

    return postfix;
}

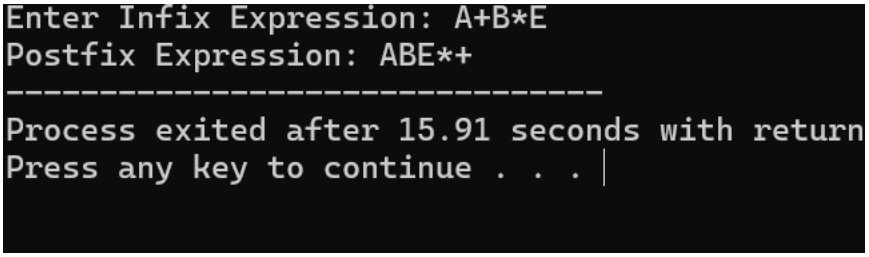
int main()
{
    string infix;
```

```
    cout << "Enter Infix Expression: ";
    getline(cin, infix);

    cout << "Postfix Expression: "
         << infixToPostfix(infix);

    return 0;
}
```

OUTPUT:



TASK#03

```
#include <iostream>
#include <string>
using namespace std;

struct Node
{
    int data;
    Node* next;
};
```

```
class Stack
{
private:
   Node* top;

public:
   Stack()
   {
      top = NULL;
   }

   void push(int value)
   {
      Node* p = new Node;
      p->data = value;
      p->next = top;
      top = p;
   }

   int pop()
   {
      if (top == NULL)
         return 0;

      Node* p = top;
```

```
        int value = p->data;
        top = top->next;
        delete p;

        return value;
    }

    bool isEmpty()
    {
        return top == NULL;
    }
};

int main()
{
    string postfix;
    int A, B, C;

    cout << "Enter value of A: ";
    cin >> A;

    cout << "Enter value of B: ";
    cin >> B;

    cout << "Enter value of C: ";
    cin >> C;
```

```
cout << "Enter Postfix Expression: ";
cin >> postfix;

Stack s;

for (int i = 0; i < postfix.length(); i++)
{
   char ch = postfix[i];

   if (ch == 'A')
   {
      s.push(A);
   }
   else if (ch == 'B')
   {
      s.push(B);
   }
   else if (ch == 'C')
   {
      s.push(C);
   }
   else
   {
      int value2 = s.pop();
      int value1 = s.pop();
```

```
            int result;

            if (ch == '+')
               result = value1 + value2;

            else if (ch == '-')
               result = value1 - value2;

            else if (ch == '*')
               result = value1 * value2;

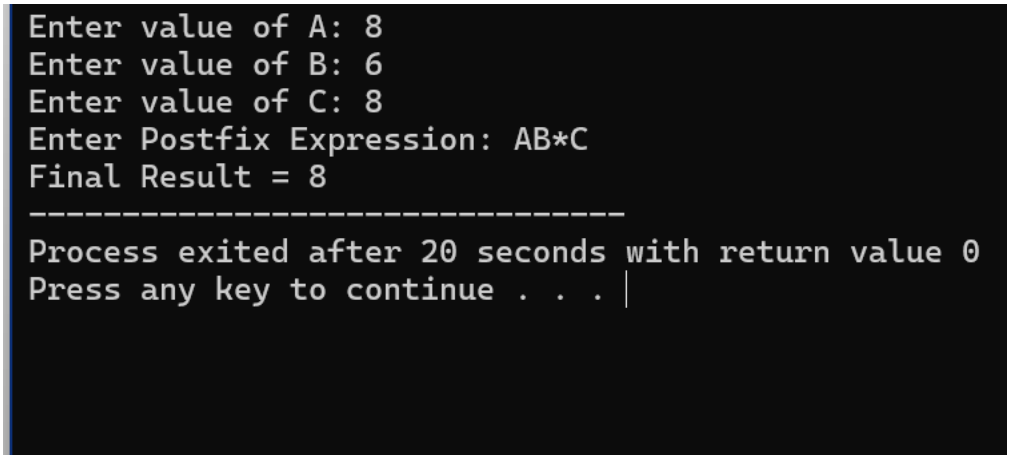
            else if (ch == '/')
               result = value1 / value2;

            s.push(result);
         }
      }

      cout << "Result = " << s.pop();

      return 0;
}
```

OUTPUT:


Enter value of A: 8
Enter value of B: 6
Enter value of C: 8
Enter Postfix Expression: AB*C
Final Result = 8

Process exited after 20 seconds with return value 0
Press any key to continue . . .